Alerts and anomaly detection: `engine/alerts.py`

Flags unusual usage, sudden cost increases, resources heading toward their limits, and projected budget overruns. Every alert carries a plain-English `why` that states the numbers behind it.

1. What it does

Inputs: the VM list, per-VM statistics (`stats.py`), raw series, daily cost per day, the projected month cost, and the recommendations per VM. **Output:** a list of alerts, each with `id`, `kind`, `severity`, a title, a `why` explanation and a `cost_impact` in ₹. Sorted by severity (critical first), then by cost impact.

Kind	Severity	Trigger (section)
anomaly	critical	Sustained abnormal CPU (2)
budget	critical	Projected month above the budget (7)
cost_jump	warning	Daily cost jumps (6)
capacity (disk)	warning	Disk reaches 90% within 21 days (3)
capacity (CPU or memory trend)	warning	Steady climb toward 90% (4)
capacity (CPU saturated)	warning	p99 CPU above 90% (5)

2. Anomaly detection (`robust_anomaly`)

The goal is to flag a VM that behaves unlike itself without flagging VMs that spike at the same time every day.

1. **Training window:** the first 70% of the data (21 of 30 days). The VM is skipped if the data window is under 10 days, or if more than half of the training readings are missing (so a VM launched late in the training window is not scored).
2. **Seasonal baseline:** for each of 48 slots (weekday or weekend × hour of day), take the **median** CPU and the **MAD** (median absolute deviation) over the training window.
3. **Scoring:** for every reading after the training window, `z = (value − slot median) ÷ max(1.4826 × MAD, 2.0)`. The 1.4826 factor makes MAD comparable to a standard deviation; the floor of 2.0 stops a very flat VM from producing huge z-scores for tiny changes. Training days are set to 0.
4. **Hot readings:** `z > 4` (high side only).
5. **Detection:** the longest run of at least 6 consecutive hot readings (6 × 5 min = 30 minutes). Only the longest run is reported.
6. **Output:** start, end, peak z and the baseline median.

A job that runs at 10:00 every day is part of the 10:00 baseline, so it is not flagged.

Alert text includes: CPU level during the anomaly versus before it, the z-score, how many

times higher network-out is than before (corroboration only, not part of the detection), the duration in hours, and the extra egress cost: `(outbound MB/s during − before) × readings × 300 s ÷ 1024 × ₹0.80 per GB`.

Effect on recommendations: `build.py` excludes anomalous VMs from rightsizing (an "investigate" card instead), so the engine does not size on incident data.

3. Disk filling up

If the disk slope (14-day linear fit, `stats.py`) is above 0.05 percentage points per day: `days left = (90 − current disk %) ÷ slope`. Alert if 0 < days left < 21.

4. CPU or memory trend

Uses `daily_trend` from `stats.py` (line through the daily p95 of the last 14 days). Alert when all hold:

- the VM has no open anomaly alert (it is already covered),
- slope ⩾ 0.3 percentage points per day,
- $R^2 \geqslant 0.6$,
- today's fitted level is below 90%,
- `days left = (90 − level) ÷ slope` is under 21.

The text adds a hint: a steady memory climb with flat traffic usually means a leak.

5. CPU saturated

p99 CPU above 90% (and no open anomaly) → warning that points to the available upsize recommendation.

6. Cost jump

For each day from day 8 onward, compare that day's cost with the **average of the previous 7 days**. The first day above +15% raises the alert (only one cost alert is created). The text lists any VMs launched that day: how many, their plan, team, region and the added monthly price. `cost_impact = (day cost − trailing average) × 30`.

Daily cost (from `build.py`) = for each resource that exists that day: VM plan price ÷ 30.4167, disk GB × ₹6 ÷ 30.4167, outbound GB that day × ₹0.80; volumes and IPs at their monthly rates ÷ 30.4167.

7. Budget overrun

`projected month = last day's cost × 30.4167`. If it exceeds the monthly budget (₹8,50,000) a critical alert shows the overrun in rupees and percent.

8. Parameters (from `config.py`)

`ANOMALY_Z` 4.0, `ANOMALY_MIN_POINTS` 6, `ANOMALY_MIN_DAYS` 10, `COST_JUMP_PCT` 15, `CAPACITY_WARN_PCT` 90, `CAPACITY_HORIZON_DAYS` 21, `TREND_DAYS` 14, `TREND_MIN_SLOPE` 0.3, `TREND_MIN_R2` 0.6, `UNDER_P99` 90, `MONTHLY_BUDGET_INR` 850000. The 70% training split, the MAD floor of 2.0 and the 0.05 disk slope are fixed in the code.

9. Sample output (my run, seed 42)

10 alerts: budget overrun (₹1,16,581 over, projected ₹9,66,581), anomaly on `prod-pay-api-07`, cost jump of 24% on 2026-10-14 (the day the 8 ML VMs launched), a memory trend on `prod-auth-02` (about 11 days to 90%), four CPU-saturated production VMs, and disk filling on `prod-pay-api-02` and `prod-indexer-02` (about 16 days).

10. Assumptions

- The first 21 days represent normal behaviour for the VM.
- A reading is only abnormal if it is higher than normal; drops are not flagged.
- Linear trends over 14 days are good enough to project 21 days ahead.

11. Limitations

1. **Upward only.** A VM that suddenly goes silent (crash) is not detected by the anomaly rule.
2. **Only CPU is scored.** Network-out is used to explain an anomaly, not to detect one.
3. **Needs history.** A VM that launched after about day 10 is not scored (the training window is mostly empty), so a brand-new VM could misbehave unnoticed.
4. **One alert per VM per rule, and one cost jump.** After the first jump, later jumps are not reported.
5. **Anomalies inside the training window** are partly absorbed by the baseline (the median and MAD are robust, but a long incident would still skew them).
6. **Simulated incident.** The injected anomaly is very strong (CPU ~97%, egress ×25), so detection is easy by design. Weaker or noisier incidents are untested.